BDDM: Auditable arXiv-to-Lean Formalization

BDDM: Auditable arXiv-to-Lean Formalization with Standards-Positive Integrity

Status: working artifact, May 2026 **Repo:** github.com/tejoker/BDDM **Lean toolchain:** v4.29.0-rc7 **Pipeline LLM:** Mistral Leanstral (`labs-leanstral-2603`) — single-LLM policy

---

1. Problem statement

LLM-driven theorem proving for arxiv research mathematics has a quiet failure mode: **bypass promotions**. A pipeline can claim a row is `FULLY_PROVEN` while the on-disk Lean body is `sorry` , or while the proof invokes an identifier that resolves to a 0-argument `axiom : Prop` but is called with arguments. Closure-rate numbers from such pipelines are inflated by an amount that nothing checks.

BDDM is built around the question: *how do we measure honest closure rates that survive adversarial verification?*

---

2. Contributions

This work makes two concrete contributions that I'd argue are independent of any specific LLM:

2.1 Standards-positive integrity audit

A multi-layer verification gate that re-checks every claimed FP/AB/IP row against the on-disk Lean source:

1. **Body-is-sorry detection** — rejects rows whose `:= by` body contains the `sorry` placeholder (in any position, including mid-body via `<;> sorry` combinators).
2. **Trivialized-statement detection** — rejects rows whose *statement* matches a known placeholder shape (`∃ x, x = expr` , `(P Q : Prop) : P ∧ Q` , reflexive conjunctions, etc.) even when the proof technically closes.
3. **Lake-validate bodies** — runs `lake env lean` and demotes any row whose theorem body line range overlaps a reported error. Catches the "axiom invocation against `axiom : Prop` declaration" class.

The audit is **mutation-test-verified**: `tests/test_audit_integrity_mutations.py` instantiates 19 known bypass patterns and asserts the audit demotes each one. An adversarial fuzzer (`scripts/audit_fuzz_mutations.py`) ran 18,000 random bypass mutations × 9 seeds = 162,000 trials with **0 escapes**.

Across the campaign trajectory, the audit demoted **71 bypass-promoted rows** from rounds where naive validators had accepted them. This is the artifact's defining number — it's what an integrity-conscious reviewer would want to see.

2.2 Paper-axiom-budget gradient

Most prior work classifies a row as binary {proven, unproven}. BDDM splits the "proven" tier:

Tier	Lake-verified body?	All gates pass?	Paper-local axiom debt?
`FULLY_PROVEN`	✓	✓	none
`AXIOM_BACKED`	✓	every gate except `no_paper_axiom_debt`	named list (see `axiom_debt` field)
`INTERMEDIARY_PROVEN`	✓	other gates failing	possibly

This is a strict partial order: `FULLY_PROVEN > AXIOM_BACKED > INTERMEDIARY_PROVEN` . The promotion-gate logic enforces it.

The intermediate AB tier matters because realistic arxiv papers always introduce paper-local notation (`Multisegment` , `n_alpha` , `S_alpha` , etc.) that have no Mathlib counterpart yet. A pipeline that closes "modulo these named axioms" is doing useful work toward the eventual full proof; refusing to acknowledge that does the field a disservice.

The `align_def` infrastructure (`Desol.AlignDef`) is the bridge from AB to FP: a single registration `register_alignment paperDef ↔ mathlibDef := proof for "paper-id"` discharges one axiom-debt entry. AB → FP promotion is automatic when all debts are aligned. Producing the alignment proofs themselves is research-Lean work, not pipeline output — that's an honest limitation, not a bug.

---

3. Empirical results

3.1 miniF2F-244 (external calibration)

Standard miniF2F test split (244 olympiad-level problems), state-MCTS mode, k=1, workers=1, Lean v4.29.0-rc7. Result:

System	miniF2F-test pass@1
Aesop (no LLM)	4.0%
LLM-Step (Llama-2)	22.0%
ReProver (GPT-4 + best-first)	27.3%
BDDM (this work)	**29.9%** (73/244)
HyperTree Proof Search (Meta)	33.0%

BDDM beats every published baseline except Meta's HyperTree, and does so under a single-LLM policy (Leanstral) with no GPU-accelerated value/policy networks. The 28-min wall-clock is competitive.

Artifact: `reproducibility/minif2f_test_244_v429rc7_results.json` .

3.2 ArXiv corpus closure

44 canonical arxiv papers (math.* across 2010–2024), 1,464 theorem-like rows total:

```
FULLY_PROVEN           20   ( 1.4%)
AXIOM_BACKED           37   ( 2.5%)
INTERMEDIARY_PROVEN     4   ( 0.3%)
UNRESOLVED            224   (15.3%)
FLAWED               1037   (70.8%)
TRANSLATION_LIMITED   142   ( 9.7%)
                     ————
                     1464
```

Honest closure rate: 61/(61+224) = **21.4%** over closure-eligible rows (FP+AB+IP vs UR).

The 71% FLAWED rate is honest failure-mode evidence — the translation acceptance gate refuses placeholder / shape-mismatched / quantifier-flipped translations rather than promoting them. This is a feature, not a bug: a translation pipeline that “succeeds” on every row is a pipeline that lies.

3.3 Non-Leanstral optimization trajectory

A multi-round campaign measured how much closure could be gained from **non-LLM** infrastructure alone, holding Leanstral fixed:

Round	Mechanism	Honest closure delta
XIII	aux deterministic micro-prover pre-pass	+12
XIV	parent deterministic pre-pass	+4
XV	larger candidate count	+3
XVI	hardened audit (lake-validate-bodies)	+2 net
XVII	expanded 20-tactic catalog	**0 — saturated**
XVIII–XIX	+ scale to new arxiv	+5
XX	additional sweep	+4
XXI	multi-shot N=8 + aesop-safe compounding	+2

Round XVII produced 0 honest gains — empirical evidence of the non-Leanstral ceiling under the current architecture. Subsequent gains came from scaling (more papers) rather than infrastructure changes on the existing papers.

3.4 Audit-survival robustness

Audit class	Bypass patterns caught	Mutation-test patterns	Fuzz iterations × escapes
Body-is-sorry + namespace	7	7/7	—
Trivialized-statement	5	5/5	—

Audit class	Bypass patterns caught	Mutation-test patterns	Fuzz iterations × escapes
Sorry mid-body / combinators	3	3/3	—
Prop-binder placeholder	1	1/1	—
Lake-validate-bodies (new)	1 class, 11 historical demotions	7/7	—
Adversarial fuzz	—	—	**162,000 × 0 escapes**

4. Architecture (one-page summary)

```
arXiv ID
   |
   ▼ [1] LaTeX preprocessor: \newcommand / \input / \subfile inlining
   ▼ [2] Theorem extractor: theorem/lemma/proposition/corollary
   ▼ [3] Translator (Leanstral): LaTeX → Lean 4 signature
        + acceptance gate: vacuity / triviality / quantifier-scope-flip refusal
   ▼ [4] Paper-theory builder: typeclass instances + [aesop safe] axioms
   ▼ [5] Premise retrieval: 220k Mathlib name index, 205k premise index
   ▼ [6] Proof search:
        (a) deterministic catalog (20 tactics, paper-tuned priors)
        (b) state-MCTS via leanprover-community/repl
        (c) lemma-factor v3 (recursive depth-2, type-aware composition)
        (d) whole-proof + REPL-driven generators (multi-shot N=8 temperature ladder)
   ▼ [7] Verification ledger
   ▼ [8] CoT alignment review (per-area domain rules)
   ▼ [9] Bridge: reviewed-equivalent → gold queue → ledger flip
   ▼ [10] Integrity audit  ← THE KEY GATE
        body-is-sorry · trivialization · lake-validate-bodies
   ▼ [11] Reproducibility-bundle mirror (committed evidence path)
   ▼ [12] Optional: align_def discharge → AB → FP promotion
```

5. Honest limitations

1. FP gain is bottlenecked by `align_def` discharge. The campaign moved FP from 14 → **20 (+6 honest)** via automated proposal of trivial alignments (script `auto_align_proposer.py` : parse paper-theory stub → infer alignment shape → lake-validate → register). The hardened audit caught 7 spurious promotions in the same pass (rows whose `axiom_debt` was discharged but whose proof body was still `sorry`) — those were demoted, leaving +6 audit-survived FP. Beyond this, `align_def` discharge for non-trivial paper-local concepts (real mathematical content, not

`:= 0` stubs) requires research-Lean work that no LLM currently automates — `paperDef ↔ MathlibDef` proofs for novel concepts are weeks-per-axiom human work or require Mathlib-side definitional growth.

2. Parent composition is 0/N every round. Lemma-factor v3 produces aux statements that close individually but whose conjunction/structure doesn't match what the parent needs. This is a binder-spatial-reasoning gap in the LLM, not a tactic-search gap. Likely needs richer type-aware composition templates and/or stronger LLM reasoning.

3. Single-LLM lock-in. The current code path uses Leanstral exclusively. A model ensemble (DeepSeek-Prover, Kimina, Llemma) would almost certainly raise closure rates by ~10–30%, at the cost of pipeline complexity. Deliberately deferred to keep variance controlled while the integrity audit was being hardened.

4. ArXiv corpus is biased toward older papers. The OAI-PMH harvest by date matches re-indexing date, not publication date; most batch papers are from 2010–2023. A modern (2024+) corpus would test the translator's robustness on contemporary notation conventions.

5. CoT judge has no human-in-the-loop yet. All `reviewer_type` values are `auto_llm`. The `release_eligible` gate, which requires `reviewer_type ∈ {human, hybrid}`, has never fired in practice. Production release would need a human review loop.

---

6. Research directions

1. **`align_def` proof generation.** Use Leanstral to *propose* `paperDef ↔ mathlibDef` alignment proofs, with the registration step gated by lake-verification. Discharges paper-axiom debt at scale.
2. **Signature-patching pre-pass.** When the per-row baseline error names `synthInstanceFailed: <Class> <FreeVar>`, splice `[<Class> <FreeVar>]` into the signature. Already implemented and behind `--use-typeclass-patcher` (default OFF — needs calibration on broader corpus).
3. **Type-aware composition synthesis.** Replace the current template-based composition emitter with one that uses each aux's elaborated *type* (from the REPL) to pick the right shape (`⟨…⟩` vs `And.intro` vs `Or.inl/inr` vs `Exists.intro` vs `Iff.intro`).
4. **Mathlib-blessed axiom-budget infrastructure.** Coordinate with Mathlib maintainers on a community-blessed `paper_local_axiom` attribute so the FP/AB distinction is durable across Mathlib versions.

---

7. Citation

```
@software{desol2026,
  title={DESol/BDDM: auditable arXiv-to-Lean formalization with standards-positive integrity},
```

```
  year={2026},
  url={https://github.com/tejoker/BDDM}
}
```